



VINUNIVERSITY

Lab 06: Sorting Algorithms.

Week 06, Apr 05 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get **0** point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.
- **Navigate to the Lab:** Open the specific link provided for this week's lab: <https://vinuni26oop.contest.codeforces.com/>.

Problem 1: Sort Students by Multiple Criteria (Merge Sort)

You are given a list of students. Each student has:

- ID (integer)
- Name (string, a single word with no spaces)
- GPA (double)

Your task is to sort the students using the Merge Sort algorithm based on the following criteria:

- Primary key: GPA in non-increasing order (higher GPA first)
- Secondary key: ID in increasing order (smaller ID first)

Input

- Line 1: Integer n (number of students)
- Next n lines: Each line contains ID, Name, GPA

Output

- Print the sorted list of students, one per line, in the format: ID Name GPA
- GPA must be printed with exactly one digit after the decimal point

Notes

- Constraints:
 - $1 \leq n \leq 10^5$
 - $0.0 \leq \text{GPA} \leq 4.0$
 - ID is a positive integer
- You must implement Merge Sort manually. Use of built-in sorting functions (e.g., `Arrays.sort`, `Collections.sort`) is not allowed.
- Merge Sort is stable. Make sure that students with the same GPA and ID maintain their original input order.

Example**Examples**

Input:

```
3
101 Alice 3.5
102 Bob 3.8
103 Carol 3.5
```

Output:

```
102 Bob 3.8
101 Alice 3.5
103 Carol 3.5
```

Problem 2: Sort Tasks by Deadline (Insertion Sort)

You are given a list of tasks. Each task has:

- Name (string, single word)
- Deadline (integer)

Your task is to sort the tasks using the Insertion Sort algorithm in non-decreasing order of deadline (earlier deadlines first).

The sorting algorithm must be **stable**: if two tasks have the same deadline, they should appear in the same relative order as in the input.

Input

- Each line contains a task name and its deadline separated by a space.
- Input ends with a line containing a single #.
- There is at least one task before the terminating symbol.

Output

- Print the sorted list of tasks, one per line, in the format: `name deadline`

Notes

- $1 \leq \text{number of tasks} \leq 10,000$
- $0 \leq \text{deadline} \leq 10,000$
- Maintain the original input order for tasks with equal deadlines (stable sort)

Example

Examples

Input:

```
TaskA 5
TaskB 2
TaskC 5
#
```

Output:

```
TaskB 2
TaskA 5
TaskC 5
```

Problem 3: Count Swaps while Sorting Objects (Bubble Sort)

You are given a list of products. Each product has:

- Name (string, a single word)
- Price (integer)

Sort the products using the *Bubble Sort* algorithm in non-decreasing order of price (lowest price first).

Count the total number of swaps performed. The sorting must be **stable**, meaning that if two products have the same price, their original relative order must be preserved.

Input

- Line 1: integer n (number of products)
- Next n lines: each line contains a product name and price separated by a space

Output

- Print the sorted list of products, one per line, in the format: `name price`
- After the list, print the total number of swaps in the format: `Swaps: x`

Notes

- Only actual swaps of elements count toward the total
- Bubble Sort is naturally stable if implemented properly
- Constraints:
 - $1 \leq n \leq 10,000$
 - $0 \leq \text{Price} \leq 10^6$

Example

Examples

Input:

3

A 30

B 10

C 20

Output:

B 10

C 20

A 30

Swaps: 2

Problem 4: QuickSort Strings by Length

Given a list of strings, sort them using *QuickSort* based on the following criteria:

- Primary: string length in ascending order (shorter strings first)
- Secondary: lexicographical order

The sorting should be stable if possible, but partition-based QuickSort may not guarantee stability.

Input

- Line 1: integer n (number of strings)
- Next n lines: each line contains a string (single word)
- Constraints:
 - $1 \leq n \leq 10^5$
 - Each string contains only printable ASCII characters
 - Length of each string ≤ 100

Output

- Print the sorted strings, one per line, following the sort criteria

Notes

- Use partition-based QuickSort; do not use built-in sorting functions
- Secondary criterion (lexicographical order) is used only when two strings have equal length

Example

Examples

Input:

```
4
pear
apple
fig
banana
```

Output:

```
fig
pear
apple
banana
```